

# Exercise 1: Generate a Full-Stack CAP Application Using Joule

## Introduction

In this exercise, you will use **Joule for Developers**  inside SAP Business Application Studio (BAS) to generate a full-stack **CAP (Cloud Application Programming model)** application with a Fiori Elements UI — from a single natural language prompt.

The application you will generate is an **Employee Discount Management System**. It allows employees to view and request discount cards, while administrators manage offers, approve requests, and handle reporting.

By the end of this exercise, you will have:

- A CAP application with a CDS data model, OData service, and Fiori Elements UI
- Auto-generated test data
- Custom validation logic added through Joule

## Exercises

- [1.1 - Create the project in BAS](#)
- [1.2 - Generate the application with Joule](#)
- [1.3 - Preview the application locally](#)
- [1.4 - Add custom logic](#)
- [1.5 - Regenerate test data](#)
- [Summary & Next exercise](#)

---

## Exercise 1.1: Create the project in BAS

[^Top of page](#)

**Open SAP Business Application Studio and set up a workspace for the exercise.**

►  Click to expand!

---

## Exercise 1.2: Generate the application with Joule

[^Top of page](#)

**Use the Fiori: Project Accelerator powered by Joule  to generate the full-stack CAP application from a natural language prompt.**

►  Click to expand!

---

## Exercise 1.3: Preview the application locally

[^Top of page](#)

**Run the CAP application locally and open the Fiori Elements preview to verify the generated UI and data.**

►  Click to expand!

---

## Exercise 1.4: Add custom logic

[^Top of page](#)

**Use the Joule `/cap-gen-logic` annotation to add server-side validation:  
`EmployeeID` must be a 10-digit number (digits only).**

►  Click to expand!

---

## Exercise 1.5: Regenerate test data

[^Top of page](#)

The original test data contains non-numeric `EmployeeID` values that now fail the validation. Use Joule to regenerate it with valid IDs.

►  Click to expand!

---

## Summary & Next exercise

[^Top of page](#)

Congratulations! In this exercise you used **Joule for Developers**  to:

- Generate a full-stack **CAP application** — entities, associations, OData service, and Fiori Elements UI — from a single natural language prompt.
- Preview and verify the application running locally in BAS.
- Add **server-side validation logic** using the Joule `/cap-gen-logic` annotation.
- Regenerate **test data** using the Joule `/cap-data-gen` annotation to match the new validation rules.

This demonstrates how Joule accelerates CAP development by automating boilerplate generation, while still letting developers guide and customize the output through natural language.

---

Continue to [Exercise 2: Generate a RAP transactional app from scratch](#).

## Exercise: Generate a transactional app from scratch using Joule

### Introduction

In this exercise, you will create a transactional UI service along with the underlying RAP business object to process *Product* entities.

First, you'll create an ABAP package, a database table, and an ABAP class to populate the database table with demo data. Using **ABAP Cloud Generators: Transactional App from Scratch** , you will input a natural language prompt describing the application, and all its artifacts will be generated. This includes the CDS data model, behavior definition, behavior implementation class, service definition, and service binding. After

that, you'll publish and check your Smart Inventory application using the SAP Fiori elements preview.

Refer to the [ABAP Cloud Roadmap Information - GenAI](#) for more details.

## Exercises

- [1.1 - Create package](#)
- [1.2 - Generate the transactional UI service](#)
- [1.3 - Adjust metadata extension](#)
- [1.4 - Publish and preview the \*Product app\*](#)
- [Summary & Next exercise](#)

### Reminder:

- Don't forget to replace all occurrences of the placeholder `###` with your group ID in the exercise steps below.
- You can use the ADT function **Replace All (Ctrl+F)** for the purpose.
- If you don't have a group ID yet, then check the section [Getting Started - Group ID](#) to define a suffix for your artifacts.

## Exercise 1.1: Create package

[^Top of page](#)

Create your exercise package  package ZWORKSHOP\_AI\_### .

This ABAP package will contain all the artifacts you will be creating in the different exercises of this hands-on session.

►  Click to expand!

## Exercise 1.2: Generate the transactional UI service

[^Top of page](#)

Generate an OData UI service using the ABAP Cloud Generator: Transactional app from scratch 

►  Click to expand!

[^Top of page](#)

## Exercise 1.4: Publish and preview the app

[^Top of page](#)

**Publish the local service endpoint of your service binding** 

**ZUI\_INVENTORYMANAGEMENT\_04### and start the *Fiori elements App Preview* in ADT.**

►  Click to expand!

## Summary & Next exercise

[^Top of page](#)

Now that you've...

- Created an ABAP package,
  - Generated a SAP Fiori app based on the ABAP RESTful Application Programming model (RAP) from scratch, Just by prompt in natural language.
  - Published a local service point, and started the Fiori elements App Preview in ADT.
- 
-